

光线追踪：下一周

彼得·雪莉、特雷弗·戴维·布莱克、中蒂夫·霍拉施

4.0.2 版本，2025 年 4 月 25 日

©2018-2024 彼得·雪莉。保留所有权利。

目录

- 1 概述
- 2 运动模糊
 - 2.1 时空光线追踪简介
 - 2.2 时间管理
 - 2.3 更新相机以模拟运动模糊
 - 2.4 添加移动球体
- 2.5 跟踪光线相交时间
- 2.6 整合所有内容
- 3 包围体层次结构
 - 3.1 核心思想
 - 3.2 包围体层次结构
 - 3.3 轴对齐边界框 (AABBs)
 - 3.4 射线与轴对齐包围盒的相交
 - 3.5 为命中对象构建边界框
 - 3.6 创建对象列表的边界框
 - 3.7 BVH 节点类
 - 3.8 分割 BVH 列表
 - 3.9 包围盒比较函数
 - 3.10 另一种 BVH 优化
- 4 纹理映射
 - 4.1 恒定颜色纹理
 - 4.2 立方体纹理：棋盘格纹理
 - 4.3 渲染立方体棋盘格纹理
 - 4.4 球体的纹理坐标
 - 4.5 访问纹理图像数据
 - 4.6 渲染图像纹理
- 5 柏林噪声
 - 5.1 使用随机数块
 - 5.2 平滑结果
 - 5.3 利用埃尔米特平滑进行改讲
 - 5.4 调整频率
 - 5.5 在晶格占卜使用随机向量
 - 5.6 引入湍流
 - 5.7 调整相位

6	四边形
6.1	定义四边形
6.2	射线与平面的相交
6.3	找到包含给定四边形的平面
6.4	平面上的点的定向
6.5	射线与平面相交
6.6	使用 UV 坐标对交点进行内部测试
6.7	其他 2D 图元
7	光源
7.1	白发光材质
7.2	为光线颜色函数添加背景色
7.3	将物体转化为光源
7.4	创建一个空的“康奈尔盒”
8	实例
8.1	实例平移
8.2	实例旋转
9	体积
9.1	恒定密度介质
9.2	渲染带有烟雾盒和雾盒的康奈尔盒
10	测试所有新功能的场景
11	致谢
12	引用本书
12.1	基本数据
12.2	代码片段
12.2.1	标记语言
12.2.2	HTML
12.2.3	LaTeX 和 BibTeX
12.2.4	RibLaTeX
12.2.5	电气和电子工程 biblatex
12.2.6	司法协助

概述

在《周末搞定光线追踪》中，你构建了一个简单的暴力路径追踪器。在本部分中，我们将添加纹理、体积（如雾气）、矩形、实例、光源，并使用 BVH 来支持大量物体。完成后，你将拥有一个“真正的”光线追踪器。

光线追踪领域有一个很多人（包括我在内）都认同的启发式原则，那就是大多数优化会让代码变得复杂，却不会带来多少速度提升。在这本小册子里，我做每一个设计决策时都会采用最简单的方法。有关更多与项目相关的资源，请参阅我们的“延伸阅读”维基页面。不过，我强烈建议大家不要进行过早优化；如果某项内容在执行时间分析中占比不高，那么在所有功能都得到支持之前，都不需要对其进行优化！

这本书中最难的两个部分是 BVH 和柏林噪声纹理。这也是为什么标题建议你花一周时间而不是一个周末来完成这项任务。不过，你可以把这些内容留到最后再处理，如果你……

想要一个周末项目。对于本书中介绍的概念来说，顺序并不是很重要，而且即使没有 BVH 和 Perlin 纹理，你仍然能做出康奈尔盒！

有关该项目的信息、GitHub 上的代码库、目录结构、构建与运行方式以及如何提出或参考修正和贡献，请参阅项目的 README 文件。

这些书籍的格式经过优化，可直接从浏览器中顺利打印。我们还在每个版本的“资源”部分提供了每本书的 PDF 文件。

感谢所有在这个项目中伸出援手的人。你可以在本书末尾的致谢部分找到他们。

运动模糊

当你决定采用光线追踪时，就意味着你认为视觉质量比运行时间更重要。在渲染模糊反射和散焦模糊时，我们每个像素使用了多个样本。一旦踏上这条路，好消息是几乎所有效果都可以用类似的暴力方法实现。运动模糊无疑就是其中之一。

在真实相机中，快门会在一段短时间内保持打开状态，在此期间，相机和现实世界中的物体可能会移动。为了精确重现这样的相机拍摄效果，我们需要计算相机快门打开时所感知到的内容的平均值。

2.1. 时空光线追踪简介

我们可以通过在快门打开时的某个随机时刻发射一条光线，来得到单个（简化的）光子的随机估计值。只要我们能确定在那一时刻物体应该在的位置，就能准确测量出同一时刻该光线的光量。这是随机（蒙特卡洛）光线追踪最终变得相当简单的又一个例子。蛮力方法再次获胜！

由于光线追踪器的“引擎”只需确保物体在每条光线所需的位置上，因此相交的核心部分不会有太大变化。要实现这一点，我们需要为每条光线存储确切的时间：

清单 1: [ray.h] 带有时间信息的光线

2.2. 时间管理

在继续之前，让我们先思考一下时间，以及我们如何在一次或多次连续渲染中对其进行管理。快门定时有两个方面需要考虑：从一次快门打开到下一次快门打开的时间，以及每帧快门保持打开的时长。标准电影胶片过去的拍摄帧率是每秒 24 帧。现代数字电影的帧率可以是 24、30、48、60、120 帧每秒，或者是导演想要的任何帧率。

每一帧都可以有自己的快门速度。这个快门速度不必是 —— 而且通常也不是 —— 整个帧的最大持续时间。你可以让每帧的快门开启 1/1000 秒，或者 1/60 秒。

如果你想渲染一系列图像，就需要为相机设置合适的快门时间：帧与帧之间的间隔、快门 / 渲染持续时间以及总帧数（总拍摄时间）。如果相机在移动而场景是静止的，这样设置就足够了。但如果场景中有任何物体在移动，你就需要为可命中对象添加一种方法，让每个物体都能知晓当前帧的时间段。这种方法将为所有动画对象提供在该帧内设置其运动状态的途径。

这相当简单明了，如果你愿意，这绝对是一个有趣的尝试方向。不过，就我们目前的目的而言，我们将采用一个

只渲染单帧，默认假设时间从 0 开始，到 1 结束。我们的首要任务是创建一个动画球体类。其次是修改相机，使其发射的光线具有 $[0,1]$ 范围内的随机时间。

2.3. 更新相机以模拟运动模糊

我们需要修改相机，使其在开始时间和结束时间之间的随机时刻生成光线。相机应该记录时间间隔，还是应该由相机的使用者在创建光线时来决定呢？如果有疑问，我倾向于让构造函数复杂一些，这样调用起来会更简单，所以我会让相机来记录时间间隔，但这只是个人偏好。相机不需要太多改动，因为目前不允许它移动，它只是在一段时间内发射光线而已。

清单 2: [camera.h] 带有时间信息的相机

2.4. 添加移动球体

现在来创建一个移动的物体。我会更新球体类，使其中心在时间为 0 时从 `center1` 沿直线移动到时间为 1 时的 `center2`。（在该时间间隔之外，它会无限继续移动，因此确实可以在任何时间对其进行采样。）我们将通过用一条 3D 射线替换 3D 中心点来实现这一点，该射线描述了时间为 0 时的初始位置以及到时间为 1 时终点位置的位

清单 3: [sphere.h] 一个移动的球体

更新后的 `sphere::hit ()` 函数与旧的 `sphere::hit ()` 函数几乎完全相同：我们现在只需要确定动画中心的当前位置。

清单 4: [sphere.h] 移动球体的相交函数

2.5. 跟踪光线相交时间

既然光线具有时间属性，我们就需要更新 `material::scatter()` 方法，以考虑相交时间：

```
class lambertian : public material {
    ...
    bool scatter(const ray& r_in, const hit_record& rec, color& attenuation, ray&
    scattered)
    const override {
        auto scatter_direction = rec.normal + random_unit_vector();

        // Catch degenerate scatter direction
        if (scatter_direction.near_zero())
            scatter_direction = rec.normal;

        scattered = ray(rec.p, scatter_direction, r_in.time());
        attenuation = albedo;
        return true;
    }
    ...
};

class metal : public material {
    ...
    bool scatter(const ray& r_in, const hit_record& rec, color& attenuation, ray&
    scattered)
    const override {
        vec3 reflected = reflect(r_in.direction(), rec.normal);
        reflected = unit_vector(reflected) + (fuzz * random_unit_vector());
        scattered = ray(rec.p, reflected, r_in.time());
        attenuation = albedo;

        return (dot(scattered.direction(), rec.normal) > 0);
    }
    ...
};

class dielectric : public material {
    ...
    bool scatter(const ray& r_in, const hit_record& rec, color& attenuation, ray&
    scattered)
    const override {
        ...
        scattered = ray(rec.p, direction, r_in.time());
        return true;
    }
    ...
};
```

清单 5: [material.h] 在 `material::scatter()` 方法中处理光线时间

2.6. 整合所有内容

下面的代码以最后一本书末尾场景中的漫反射球体为例，以及时间 $t = 1$ 的 $C + (0, r/2, 0)$ ：让它们在图像渲染过程中移动。每个球体从时间 $C_t = 0$ 的中心开始移动到

```
int main() {
    hittable_list world;

    auto ground_material = make_shared<lambertian>(color(0.5, 0.5, 0.5));
    world.add(make_shared<sphere>(point3(0,-1000,0), 1000, ground_material));

    for (int a = -11; a < 11; a++) {
        for (int b = -11; b < 11; b++) {
            auto choose_mat = random_double();
            point3 center(a + 0.9*random_double(), 0.2, b + 0.9*random_double());

            if ((center - point3(4, 0.2, 0)).length() > 0.9) {
                shared_ptr<material> sphere_material;

                if (choose_mat < 0.8) {
                    // diffuse
                    auto albedo = color::random() * color::random();
                    sphere_material = make_shared<lambertian>(albedo);
                    auto center2 = center + vec3(0, random_double(0,.5), 0);
                    world.add(make_shared<sphere>(center, center2, 0.2,
sphere_material));
                } else if (choose_mat < 0.95) {
                    ...
                }
            }
            ...

            camera cam;

            cam.aspect_ratio      = 16.0 / 9.0;
            cam.image_width       = 400;
            cam.samples_per_pixel = 100;
            cam.max_depth         = 50;

            cam.vfov              = 20;
            cam.lookfrom = point3(13,2,3);
            cam.lookat  = point3(0,0,0);
            cam.vup      = vec3(0,1,0);

            cam.defocus_angle = 0.6;
            cam.focus_dist    = 10.0;

            cam.render(world);
        }
    }
}
```

清单 6: [main.cc] 上一本书的最终场景，但包含移动的球体

这给出了以下结果：

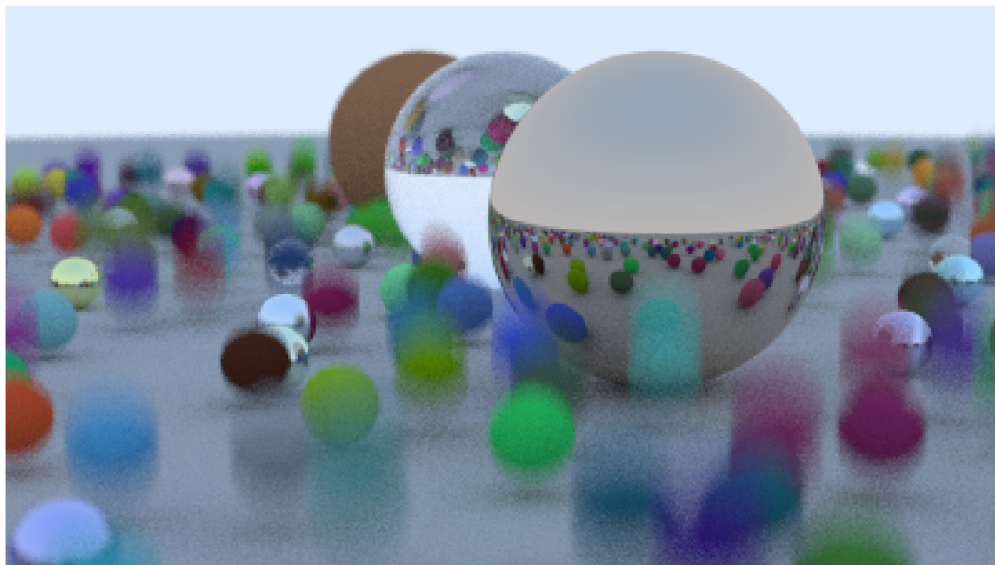


图 1：弹跳球体

包围体层次结构

到目前为止，这部分是我们正在开发的光线追踪器中最困难且最复杂的部分。我把它放在这一章，这样代码可以运行得更快，而且因为它对可命中对象进行了一些重构，当我添加矩形和立方体时，我们就不必回头去重构它们了。

光线与物体的相交计算是光线追踪器中的主要时间瓶颈，其运行时间与物体数量成线性关系。但这是在同一场景中进行的重复搜索，因此从二分查找的思路出发，我们应该能够将其转化为对数级搜索。由于我们要向同一场景发射数百万到数十亿条光线，我们可以对场景中的物体进行排序，这样每条光线的相交计算就能成为亚线性搜索。两种最常见的排序方法是：1) 对空间进行细分；2) 对物体进行细分。后者通常更易于编码，并且对于大多数模型来说，运行速度也不相上下。

3.1. 核心思想

为一组图元创建包围体的核心思想是找到一个能完全包围（界定）所有对象的体积。例如，假设你计算出了一个包围 10 个对象的球体。任何未命中这个包围球的射线，必然会错过球内的所有 10 个对象。如果射线命中了包围球，那么它就有可能命中这 10 个对象中的一个。因此，包围体代码的形式通常如下：

```
if (ray hits bounding object)
    return whether ray hits bounded objects
else
    return false
```

请注意，我们将使用这些包围体将场景中的对象分组为子组。我们不会划分屏幕或场景空间。我们希望任何给定的对象只存在于一个包围体中，不过包围体可以重叠。

3.2. 包围体层次结构

为了实现亚线性，我们需要让包围体具有层级结构。例如，如果我们将一组对象分为两组，红色组和蓝色组，并使用矩形包围体，那么我们会得到：

图 1：包围体层次结构

请注意，蓝色和红色的包围体包含在紫色包围体中，但它们可能会重叠，并且没有顺序——它们都只是在内部。因此，右侧所示的树对左右子节点没有顺序概念；它们只是在内部。代码如下：

3.3. 轴对齐包围盒（AABBs）

要让这一切顺利运作，我们需要一种方法来进行合理的分割而非不合理的分割，还需要一种让射线与包围体相交的方法。射线与包围体的相交运算需要快速，且包围体需要相当紧凑。在实践中，对于大多数模型，轴对齐的盒子比其他替代方案（如上面提到的球形边界）效果更好，但如果你遇到其他类型的包围模型，这种设计选择始终是需要考虑的因素。

从现在开始，我们将把轴对齐的边界直平行六面体（说真的，如果要精确的话，它们确实应该叫这个名字）称为轴对齐边界框，即 AABB。（在代码中，你还会遇到“bounding box”的缩写“bbox”。）任何你想用的用于计算射线与 AABB 相交的方法都是可行的。我们只需要知道是否相交即可，不需要像显示物体时那样需要求交点、法线或其他任何信息。

大多数人使用“平板”法。这是基于这样一种观察： n 维轴对齐 bounding box (AABB) 只是 n 个轴对齐区间的交集，这些区间通常被称为“平板”。回想一下，区间就是两个端点之间的点，例如，满足 $3 \leq x \leq 5$ 的 x ，或者更简洁地说 x 。在二维中，AABB（矩形）由两个区间 $[3, 5]$ 的重叠部分定义。

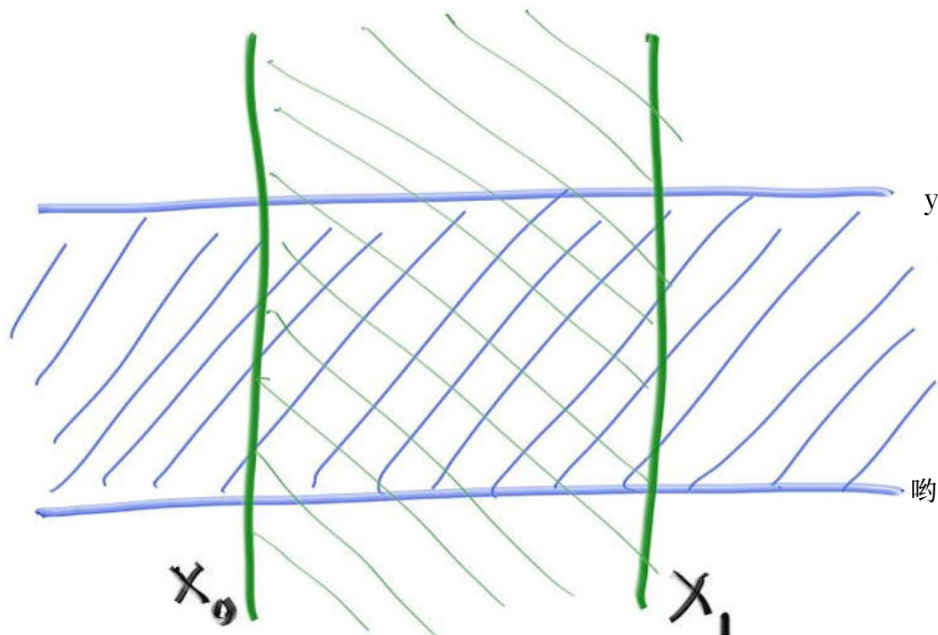


图 2：二维轴对齐边界框

要确定一条射线是否与某个区间相交，我们首先需要弄清楚这条射线是否与边界相交。例如，在一维情况下，射线与两个平面的交点会产生射线参数 t_0 和 t_1 。（如果射线与这些平面平行，那么它与任何平面的交点都是未定义的。）

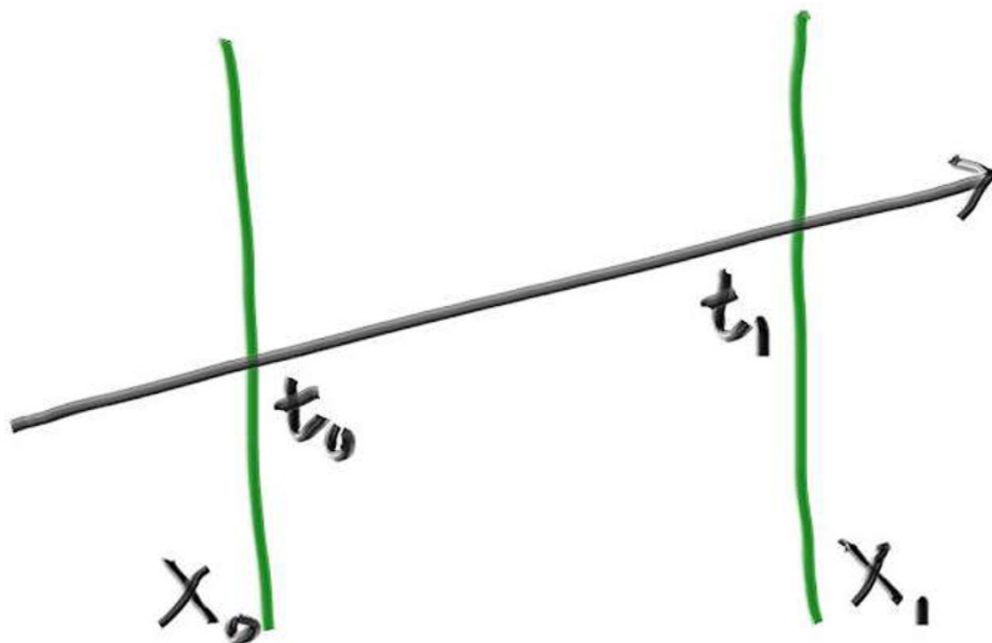


图 3：光线 - 平板相交

我们如何找到射线和平面的交点？回想一下，射线只是由一个函数定义的，该函数在给定参数 t 时会返回一个位置 $P(t)$ ：

$$P(t) = O + td$$

这个方程适用于 1 到 3 个 x, y, z 坐标。例如， $p(t) = O_x + td_x$ 。这条射线在满足以下方程的参数 t 处与平面 $x = x_0$ 相交：

$$x_0 = O_x + t d_x$$

所以 t 所以交点由下式给出

$$t_0 = \frac{x_0 - O_x}{d_x}$$

对于 x_1 ，我们得到了类似的表达式：

$$t_1 = \frac{x_1 - O_x}{d_x}$$

将一维数学转化为二维或三维命中测试的关键观察结果是：如果一条射线与盒子相交，那么只有当射线与有界盒子（由所有成对平面界定）相交时，蓝色重叠才会发生，此时所有区间都会重叠。例如，在二维中，绿色和

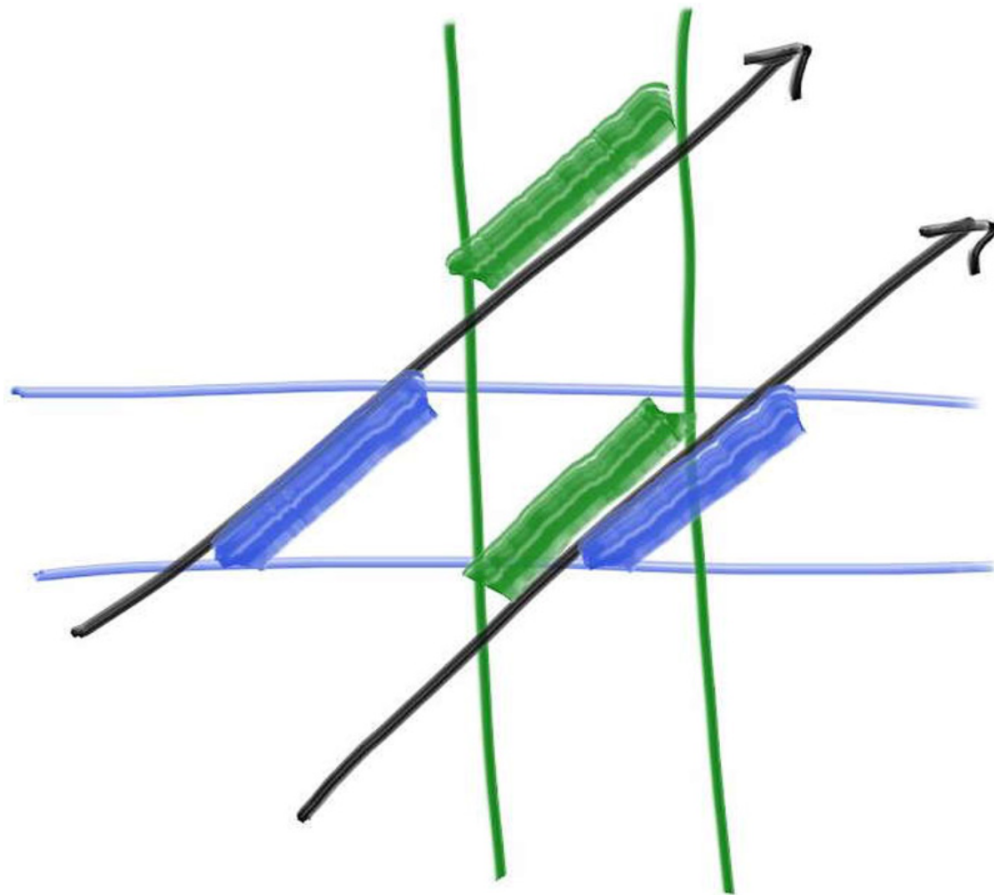


图 4：光线 - 平板 - 区间重叠 t

在该图中，上方的射线区间没有重叠，因此我们知道这条射线不会与由绿色和蓝色平面界定的二维盒子相交。下方的射线区间存在重叠，所以我们知道下方的射线会与这个有界盒子相交。

3.4. 射线与轴对齐 bounding box (AABB) 的相交检测

以下伪代码用于判断平板中的 t 区间是否重叠：

```
interval_x ← compute_intersection_x (ray, x0, x1)
interval_y ← compute_intersection_y (ray, y0, y1)
return overlaps(interval_x, interval_y)
```

这极其简单，而 3D 版本能轻松扩展上述内容，这正是人们喜欢平板法的原因：

```
interval_x ← compute_intersection_x (ray, x0, x1)
interval_y ← compute_intersection_y (ray, y0, y1)
interval_z ← compute_intersection_z (ray, z0, z1)
return overlaps(interval_x, interval_y, interval_z)
```

有一些注意事项使得这并不像最初看起来那么完美。再次考虑 t_0 和 t_1 的一维方程：

$$t_0 = \frac{x_0 - Q_x}{d_x}$$

$$t_1 = \frac{x_1 - Q_x}{d_x}$$

首先，假设光线沿负方向传播。计算得出的区间 $x(t_0, t_1)$ 为

上面的情况可能会反过来。例如，其次，分母 (7.3) d_x 可能为零。

产生无限多个值。而且，如果射线原点位于某个平板边界上，我们可能会得到一个 NaN（非数值），因为分子和分母都可能为零。此外，在使用 IEEE 浮点运算时，这个零会带有正负号。

所以，使用最小值和最大值应该能给我们正确的答案：对于 $d_x = 0$ 来说，好消息是 t_{x0} 和 t_{x1} 将会相等：要么都是 $+\infty$ ，要么都是 $-\infty$ ，如果不在 x_0 和 x_1 之间的话

$$t_{x0} = \min \left(\frac{x_0 - Q_x}{d_x}, \frac{x_1 - Q_x}{d_x} \right)$$

$$t_{x1} = \max \left(\frac{x_0 - Q_x}{d_x}, \frac{x_1 - Q_x}{d_x} \right)$$

$x_1 - Q_x = 0$ ，所以我们得到一个 NaN。在这种情况下，我们可以随意将其解释为命中或未命中，但我们稍后会重新讨论这个问题。如果我们这样做，剩下的麻烦情况是如果 $d_x = 0$ ，以及 $x_0 - Q_x = 0$ 中的任意一个。

现在，让我们来看伪函数 overlaps。假设我们可以假定这些区间没有颠倒，并且当区间重叠时，我们希望返回 true。布尔函数 overlaps() 用于计算区间 t_interval1 和 t_interval2 的重叠部分，若重叠部分非空，则确定它们是否重叠。

```
bool overlaps(t_interval1, t_interval2)
    t_min ← max(t_interval1.min, t_interval2.min)
    t_max ← min(t_interval1.max, t_interval2.max)
    return t_min < t_max
```

如果存在任何 NaN 值，比较结果将返回 false，因此如果我们在意边缘相切的情况（我们或许应该在意，因为在光线追踪器中，所有情况最终都会出现），就需要确保我们的边界框有一些填充。

为了实现这一点，我们首先会添加一个新的区间函数 `expand`，它会将一个区间按给定的量进行扩展：

```
class interval {
public:
    ...
    double clamp(double x) const {
        if (x < min) return min;
        if (x > max) return max;
        return x;
    }

    interval expand(double delta) const {
        auto padding = delta/2;
        return interval(min - padding, max + padding);
    }

    static const interval empty, universe;
};
```

清单 7: [interval.h] `interval::expand()` 方法

现在我们拥有实现新的 ABB 类所需的一切。

```

#ifndef AABB_H
#define AABB_H

class aabb {
public:
    interval x, y, z;

    aabb() {} // The default AABB is empty, since intervals are empty by default.

    aabb(const interval& x, const interval& y, const interval& z)
        : x(x), y(y), z(z) {}

    aabb(const point3& a, const point3& b) {
        // Treat the two points a and b as extrema for the bounding box, so we don't
        require a
        // particular minimum/maximum coordinate order.

        x = (a[0] <= b[0]) ? interval(a[0], b[0]) : interval(b[0], a[0]);
        y = (a[1] <= b[1]) ? interval(a[1], b[1]) : interval(b[1], a[1]);
        z = (a[2] <= b[2]) ? interval(a[2], b[2]) : interval(b[2], a[2]);
    }

    const interval& axis_interval(int n) const {
        if (n == 1) return y;
        if (n == 2) return z;
        return x;
    }

    bool hit(const ray& r, interval ray_t) const {
        const point3& ray_orig = r.origin();
        const vec3& ray_dir = r.direction();

        for (int axis = 0; axis < 3; axis++) {
            const interval& ax = axis_interval(axis);
            const double adinv = 1.0 / ray_dir[axis];

            auto t0 = (ax.min - ray_orig[axis]) * adinv;
            auto t1 = (ax.max - ray_orig[axis]) * adinv;

            if (t0 < t1) {
                if (t0 > ray_t.min) ray_t.min = t0;
                if (t1 < ray_t.max) ray_t.max = t1;
            } else {
                if (t1 > ray_t.min) ray_t.min = t1;
                if (t0 < ray_t.max) ray_t.max = t0;
            }

            if (ray_t.max <= ray_t.min)
                return false;
        }
        return true;
    }
};

#endif

```

清单 8: [aabb.h] 轴对齐边界框类

3.5. 为可命中对象构建边界框

我们现在需要添加一个函数来计算所有可命中物体的边界框。然后，我们将在所有图元上建立一个框的层次结构，而像球体这样的单个图元将位于叶节点处。

回想一下，不带参数构造的区间值默认是空的。由于 aabb 对象在其三个维度中各有一个区间，因此每个区间默认都是空的，所以 aabb 对象默认也是空的。因此，有些对象可能具有空的包围体。例如，考虑一个没有子对象的 hittable_list 对象。幸运的是，我们设计的区间类的方式使得所有数学运算都能正常进行。

最后，要记住有些对象可能是可动画的。这类对象应返回其在整个运动范围内（从时间 = 0 到时间 = 1）的边界。

```
#include "aabb.h"

class material;

...

class hittable {
public:
    virtual ~hittable() = default;

    virtual bool hit(const ray& r, interval ray_t, hit_record& rec) const = 0;

    virtual aabb bounding_box() const = 0;
};
```

清单 9: [hittable.h] 带有边界框的可命中类

对于静止的球体，bounding_box 函数很简单：

```

class sphere : public hittable {
public:
    // Stationary Sphere
    sphere(const point3& static_center, double radius, shared_ptr<material> mat)
        : center(static_center, vec3(0,0,0)), radius(std::fmax(0,radius)), mat(mat)
    {
        auto rvec = vec3(radius, radius, radius);
        bbox = aabb(static_center - rvec, static_center + rvec);
    }

    ...

    aabb bounding_box() const override { return bbox; }

private:
    ray center;
    double radius;
    shared_ptr<material> mat;
    aabb bbox;

    ...
};

```

清单 10: [sphere.h] 带边界框的球体

对于一个运动的球体，我们需要确定其整个运动范围的边界。要做到这一点，我们可以获取该球体在时间为 0 时的包围盒和在时间为 1 时的包围盒，然后计算这两个包围盒的外围包围盒。

```

class sphere : public hittable {
public:
    ...

    // Moving Sphere
    sphere(const point3& center1, const point3& center2, double radius,
           shared_ptr<material> mat)
        : center(center1, center2 - center1), radius(std::fmax(0,radius)), mat(mat)
    {
        auto rvec = vec3(radius, radius, radius);
        aabb box1(center.at(0) - rvec, center.at(0) + rvec);
        aabb box2(center.at(1) - rvec, center.at(1) + rvec);
        bbox = aabb(box1, box2);
    }

    ...
};

```

清单 11: [sphere.h] 带边界框的移动球体

现在我们需要一个新的 aabb 构造函数，它接收两个盒子作为输入。首先，我们将添加一个新的区间构造函数来实现这一点：

```
class interval {
public:
    double min, max;

    interval() : min(+infinity), max(-infinity) {} // Default interval is empty

    interval(double _min, double _max) : min(_min), max(_max) {}

    interval(const interval& a, const interval& b) {
        // Create the interval tightly enclosing the two input intervals.
        min = a.min <= b.min ? a.min : b.min;
        max = a.max >= b.max ? a.max : b.max;
    }

    double size() const {
        ...
    }
};
```

清单 12: [interval.h] 由两个区间构造的区间构造函数

现在我们可以用它从两个输入框构建一个轴对齐的边界框。

```
class aabb {
public:
    ...

    aabb(const point3& a, const point3& b) {
        ...
    }

    aabb(const aabb& box0, const aabb& box1) {
        x = interval(box0.x, box1.x);
        y = interval(box0.y, box1.y);
        z = interval(box0.z, box1.z);
    }

    ...
};
```

清单 13: [aabb.h] 从两个 AABB 输入构建的 AABB 构造函数

3.6. 创建对象列表的边界框

现在我们将更新可命中列表对象，计算其子对象的边界。我们会在添加每个新子对象时逐步更新边界框。

```
#include "aabb.h"
#include "hitable.h"

#include <vector>

class hittable_list : public hittable {
public:
    std::vector<shared_ptr<hitable>> objects;

    ...
    void add(shared_ptr<hitable> object) {
        objects.push_back(object);
        bbox = aabb(bbox, object->bounding_box());
    }

    bool hit(const ray& r, interval ray_t, hit_record& rec) const override {
        ...
    }

    aabb bounding_box() const override { return bbox; }

private:
    aabb bbox;
};
```

清单 14: [hitable_list.h] 带边界框的可命中列表

3.7. BVH 节点类

BVH 也是一种可命中对象——就像可命中对象列表一样。它实际上是一个容器，但能够响应“这条射线是否命中你？”这样的查询。一个设计问题是，我们需要两个类，一个用于树，另一个用于树中的节点；还是只需要一个类，让根节点成为我们指向的节点。命中函数相当简单：检查是否命中该节点的包围盒，如果命中，则检查子节点并理清所有细节。

在可行的情况下，我倾向于采用单类设计。以下就是这样一个类：

```
#ifndef BVH_H
#define BVH_H

#include "aabb.h"
#include "hittable.h"
#include "hittable_list.h"

class bvh_node : public hittable {
public:
    bvh_node(hittable_list list) : bvh_node(list.objects, 0, list.objects.size()) {
        // There's a C++ subtlety here. This constructor (without span indices)
        // creates an implicit copy of the hittable list, which we will modify. The lifetime of
        // the copied list only extends until this constructor exits. That's OK, because we only
        // need to persist the resulting bounding volume hierarchy.
    }

    bvh_node(std::vector<shared_ptr<hittable>>& objects, size_t start, size_t end) {
        // To be implemented later.
    }

    bool hit(const ray& r, interval ray_t, hit_record& rec) const override {
        if (!bbox.hit(r, ray_t))
            return false;

        bool hit_left = left->hit(r, ray_t, rec);
        bool hit_right = right->hit(r, interval(ray_t.min, hit_left ? rec.t :
            ray_t.max), rec);

        return hit_left || hit_right;
    }

    aabb bounding_box() const override { return bbox; }

private:
    shared_ptr<hittable> left;
    shared_ptr<hittable> right;
    aabb bbox;
};

#endif
```

清单 15: [bvh.h] 边界体积层次结构

3.8. 分割 BVH 体素

任何高效结构（包括 BVH）中最复杂的部分都是构建它。我们在构造函数中完成这一操作。BVH 的一个优点是，只要 `bvh_node` 中的对象列表

分成两个子列表，命中函数就会起作用。如果分割得当，两个子列表的边界框比其父级的边界框小，那么效果会最好，但这是为了提高速度而非保证正确性。我会选择中间方案，在每个节点处沿一个轴分割列表。我追求简单性：

随机选择一个轴

对图元进行排序（使用 `std::sort`）

每个子树中放入一半

当传入的列表有两个元素时，我会在每个子树中各放一个元素，然后结束递归。遍历算法应该流畅，不必检查空指针，所以如果只有一个元素，我会在每个子树中都复制这个元素。显式检查三个元素的情况并只进行一次递归可能会略有帮助，但我认为整个方法之后会得到优化。下面的代码使用了三个方法——`box_x_compare`、`box_y_compare` 和 `box_z_compare`，这些方法我们尚未定义。

```
#include "aabb.h"
#include "hitable.h"
#include "hitable_list.h"

#include <algorithm>

class bvh_node : public hitable {
public:
    ...

    bvh_node(std::vector<shared_ptr<hitable>>& objects, size_t start, size_t end) {
        int axis = random_int(0,2);

        auto comparator = (axis == 0) ? box_x_compare
                                : (axis == 1) ? box_y_compare
                                                : box_z_compare;

        size_t object_span = end - start;

        if (object_span == 1) {
            left = right = objects[start];
        } else if (object_span == 2) {
            left = objects[start];
            right = objects[start+1];
        } else {
            std::sort(std::begin(objects) + start, std::begin(objects) + end,
comparator);

            auto mid = start + object_span/2;
            left = make_shared<bvh_node>(objects, start, mid);
            right = make_shared<bvh_node>(objects, mid, end);
        }

        bbox = aabb(left->bounding_box(), right->bounding_box());
    }

    ...
};
```

清单 16: [bvh.h] 边界体积层次节点

这使用了一个新函数: `random_int()`:

```
...  
  
inline double random_double(double min, double max) {  
    // Returns a random real in [min,max).  
    return min + (max-min)*random_double();  
}  
  
inline int random_int(int min, int max) {  
    // Returns a random integer in [min,max].  
    return int(random_double(min, max+1));  
}  
  
...
```

清单 17: [rtweekend.h] 一个返回指定范围内随机整数的函数

检查是否存在边界框，是为了应对你传入类似无限平面这种没有边界框的情况。我们目前没有这类图元，所以在你添加这类东西之前，这种情况应该不会发生。

3.9. 箱体比较函数

现在我们需要实现用于 `std::sort()` 的盒子比较函数。为此，创建一个通用比较器，当给定一个额外的轴索引参数时，如果第一个参数小于第二个参数，该比较器返回 `true`。然后定义使用该通用比较函数的特定于轴的比较函数。

```
class bvh_node : public hittable {
    ...

private:
    shared_ptr<hittable> left;
    shared_ptr<hittable> right;
    aabb bbox;

    static bool box_compare(
        const shared_ptr<hittable> a, const shared_ptr<hittable> b, int axis_index
    ) {
        auto a_axis_interval = a->bounding_box().axis_interval(axis_index);
        auto b_axis_interval = b->bounding_box().axis_interval(axis_index);
        return a_axis_interval.min < b_axis_interval.min;
    }

    static bool box_x_compare (const shared_ptr<hittable> a, const
shared_ptr<hittable> b) {
        return box_compare(a, b, 0);
    }

    static bool box_y_compare (const shared_ptr<hittable> a, const
shared_ptr<hittable> b) {
        return box_compare(a, b, 1);
    }

    static bool box_z_compare (const shared_ptr<hittable> a, const
shared_ptr<hittable> b) {
        return box_compare(a, b, 2);
    }
};
```

清单 18: lbvh.h 的 BVH 比较函数。X 轴

到目前为止，我们已经准备好使用新的 BVH 代码了。让我们在随机球体场景中使用它。

```
#include "rtweekend.h"

#include "bvh.h"
#include "camera.h"
#include "hittable.h"
#include "hittable_list.h"
#include "material.h"
#include "sphere.h"

int main() {
    ...

    auto material2 = make_shared<lambertian>(color(0.4, 0.2, 0.1));
    world.add(make_shared<sphere>(point3(-4, 1, 0), 1.0, material2));

    auto material3 = make_shared<metal>(color(0.7, 0.6, 0.5), 0.0);
    world.add(make_shared<sphere>(point3(4, 1, 0), 1.0, material3));

    world = hittable_list(make_shared<bvh_node>(world));

    camera cam;

    ...
}
```

清单 19: [main.cc] 随机球体, 使用 BVH

渲染出的图像应与图 1 中所示的非 BVH 版本完全相同。不过, 如果你对这两个版本进行计时, BVH 版本应该会更快。我发现它的速度几乎是之前版本的 6.5 倍。

3.10. 另一种 BVH 优化

我们可以进一步加快 BVH 优化的速度。不要选择随机的分割轴, 而是分割包围盒的最长轴, 以获得最多的细分。这种修改很简单, 但在此过程中我们需要向 aabb 类添加一些内容。

第一项任务是在 BVH 构造器中构建一个与轴对齐的包围盒, 用于包围一系列物体。基本上, 我们会通过将包围盒初始化为空 (稍后我们会定义 `aabb::empty`), 然后用这一系列物体中的每个包围盒来扩展它, 从而构建 `bvh_node` 的包围盒。

一旦我们有了边界框, 就将分割轴设置为最长边所在的轴。我们可以设想一个函数来为我们完成这项工作: `aabb::longest_axis()`。最后, 由于我们预先计算了对象范围的边界框, 因此可以删除原来那条通过左右两边的并集来计算边界框的代码行。

```

class bvh_node : public hittable {
public:
    ...
    bvh_node(std::vector<shared_ptr<hittable>>& objects, size_t start, size_t end) {
        // Build the bounding box of the span of source objects.
        bbox = aabb::empty();
        for (size_t object_index=start; object_index < end; object_index++)
            bbox = aabb(bbox, objects[object_index]->bounding_box());

        int axis = bbox.longest_axis();

        auto comparator = (axis == 0) ? box_x_compare
                                : (axis == 1) ? box_y_compare
                                                : box_z_compare;

        ...

        bbox = aabb(left->bounding_box(), right->bounding_box());
    }

    ...
}

```

清单 20: [bvh.h] 为 BVH 对象范围构建边界框

现在来实现空的 aabb 代码和新的 aabb::longest_axis() 函数:

```

class aabb {
public:
    ...

    bool hit(const ray& r, interval ray_t) const {
        ...
    }

    int longest_axis() const {
        // Returns the index of the longest axis of the bounding box.

        if (x.size() > y.size())
            return x.size() > z.size() ? 0 : 2;
        else
            return y.size() > z.size() ? 1 : 2;
    }

    static const aabb empty, universe;
};

const aabb aabb::empty = aabb(interval::empty, interval::empty,
interval::empty);
const aabb aabb::universe = aabb(interval::universe, interval::universe,
interval::universe);

```

清单 21: [aabb.h] 新的 aabb 常量和 longest_axis() 函数

和之前一样，你应该会看到与图 1 相同的结果，但渲染速度会快一点。在我的系统上，这能带来大约 18% 的额外渲染速度提升。做这么一点额外的工作，效果还不错。

纹理映射

计算机图形学中的纹理映射是将材质效果应用于场景中物体的过程。“纹理”部分指的是效果，而“映射”部分在数学意义上是将一个空间映射到另一个空间。这种效果可以是任何材质属性：颜色、光泽度、凹凸几何（称为凹凸映射），甚至是材质的存在性（用于创建表面的镂空区域）。

最常见的纹理映射类型是将图像映射到物体表面，定义物体表面每个点的颜色。在实际操作中，我们会反向执行这一过程：给定物体上的某个点，我们会查找纹理映射所定义的颜色。

首先，我们将使纹理颜色具有程序性，并创建一个纯色的纹理映射。大多数程序将恒定的 RGB 颜色和纹理放在不同的类中，所以你完全可以采用不同的做法，但我非常相信这种架构，因为它能将任何颜色都转化为纹理，这一点非常棒。

为了执行纹理查找，我们需要一个纹理坐标。这个坐标可以通过多种方式定义，我们会在后续的内容中深入探讨这个概念。目前，我们将完整传入两个坐标。不过，其他类型的纹理也需要这些坐标，所以我们将它们保留在方法接口中。按照惯例，纹理坐标被命名为 u 和 v 。对于恒定纹理，每对 (u, v) 都会产生恒定的颜色，因此我们实际上可以忽略这些坐标。

我们纹理类的主要方法是 `color value (...)` 方法，该方法返回后者。给定输入坐标的纹理颜色。除了获取点的纹理坐标 u 和 v 之外，我们还提供了相关点的位置，其原因稍后会变得明确。

4.1. 恒定颜色纹理

```

#ifndef TEXTURE_H
#define TEXTURE_H

class texture {
public:
    virtual ~texture() = default;

    virtual color value(double u, double v, const point3& p) const = 0;
};

class solid_color : public texture {
public:
    solid_color(const color& albedo) : albedo(albedo) {}

    solid_color(double red, double green, double blue) :
solid_color(color(red,green,blue)) {}

    color value(double u, double v, const point3& p) const override {
        return albedo;
    }

private:
    color albedo;
};

#endif

```

清单 22: [texture.h] 一个纹理类

生命值。我们需要更新 hit_record 结构来存储射线与物体的 u 、 v 表面坐标。

```

class hit_record {
public:
    vec3 p;
    vec3 normal;
    shared_ptr<material> mat;
    double t;
    double u;
    double v;
    bool front_face;

    ...

```

清单 23: [hittable.h] 向 hit_record 添加坐标 u 、 v

可被击中的。稍后会详细说明。将来，我们需要为每种类型上的特定点计算 (u, v) 纹理坐标。

4.2. 实体纹理：棋盘格纹理

一种实体（或空间）纹理仅取决于 3D 空间中每个点的位置。你可以把实体纹理想象成它在为空间中的所有点本身着色，而不是为该空间中的某个特定对象着色。因此，当物体改变位置时，它可以在纹理的各种颜色中穿行，不过通常你会固定物体和实体纹理之间的关系。

为了探索空间纹理，我们将实现一个空间棋盘纹理类，该类用于实现三维棋盘图案。由于空间纹理函数由空间中的特定位置驱动，因此纹理的 `value()` 函数会忽略 `u` 和 `v` 参数，只使用 `p` 参数。

为了实现棋盘图案，我们首先要计算输入点每个分量的 `floor` 值。我们可以截断坐标，但那样会将值向零靠近，这会导致零的两侧呈现相同的颜色。`floor` 函数总会将值移向左侧的整数（朝着负无穷方向）。根据这三个整数结果 (`|u|`, `|z|`)，我们取它们的和并计算结果对 2 取模，结果要么是 0 要么是 1。0 对应偶数颜色，1 对应奇数颜色。

最后，我们向纹理添加一个缩放因子，以便能够控制场景中棋盘图案的大小。

```
class checker_texture : public texture {
public:
    checker_texture(double scale, shared_ptr<texture> even, shared_ptr<texture> odd)
        : inv_scale(1.0 / scale), even(even), odd(odd) {}

    checker_texture(double scale, const color& c1, const color& c2)
        : checker_texture(scale, make_shared<solid_color>(c1), make_shared<solid_color>(c2)) {}

    color value(double u, double v, const point3& p) const override {
        auto xInteger = int(std::floor(inv_scale * p.x()));
        auto yInteger = int(std::floor(inv_scale * p.y()));
        auto zInteger = int(std::floor(inv_scale * p.z()));

        bool isEven = (xInteger + yInteger + zInteger) % 2 == 0;

        return isEven ? even->value(u, v, p) : odd->value(u, v, p);
    }

private:
    double inv_scale;
    shared_ptr<texture> even;
    shared_ptr<texture> odd;
};
```

清单 24: [texture.h] 棋盘纹理

这些棋盘格奇偶参数可以指向一个恒定纹理或其他一些程序纹理。这符合帕特·汉拉汉在 20 世纪 80 年代提出的着色器网络的理念。

为了支持程序化纹理，我们将扩展朗伯体类，使其适用于纹理而非颜色：

```
#include "hitable.h"
#include "texture.h"

...

class lambertian : public material {
public:
    lambertian(const color& albedo) : tex(make_shared<solid_color>(albedo)) {}
    lambertian(shared_ptr<texture> tex) : tex(tex) {}

    bool scatter(const ray& r_in, const hit_record& rec, color& attenuation, ray&
scattered)
    const override {
        auto scatter_direction = rec.normal + random_unit_vector();

        // Catch degenerate scatter direction
        if (scatter_direction.near_zero())
            scatter_direction = rec.normal;

        scattered = ray(rec.p, scatter_direction, r_in.time());
        attenuation = tex->value(rec.u, rec.v, rec.p);
        return true;
    }

private:
    shared_ptr<texture> tex;
};
```

清单 25: [material.h] 带纹理的朗伯材质

如果我们将它添加到主场景中：

```
#include "rtweekend.h"

#include "bvh.h"
#include "camera.h"
#include "hitable.h"
#include "hitable_list.h"
#include "material.h"
#include "sphere.h"
#include "texture.h"

int main() {
    hitable_list world;

    auto checker = make_shared<checker_texture>(0.32, color(.2, .3, .1), color(.9, .9, .9));
    world.add(make_shared<sphere>(point3(0,-1000,0), 1000, make_shared<lambertian>(checker)));

    for (int a = -11; a < 11; a++) {
        ...
    }
}
```

清单 26: [main.cc] 正在使用的棋盘格纹理

我们得到：

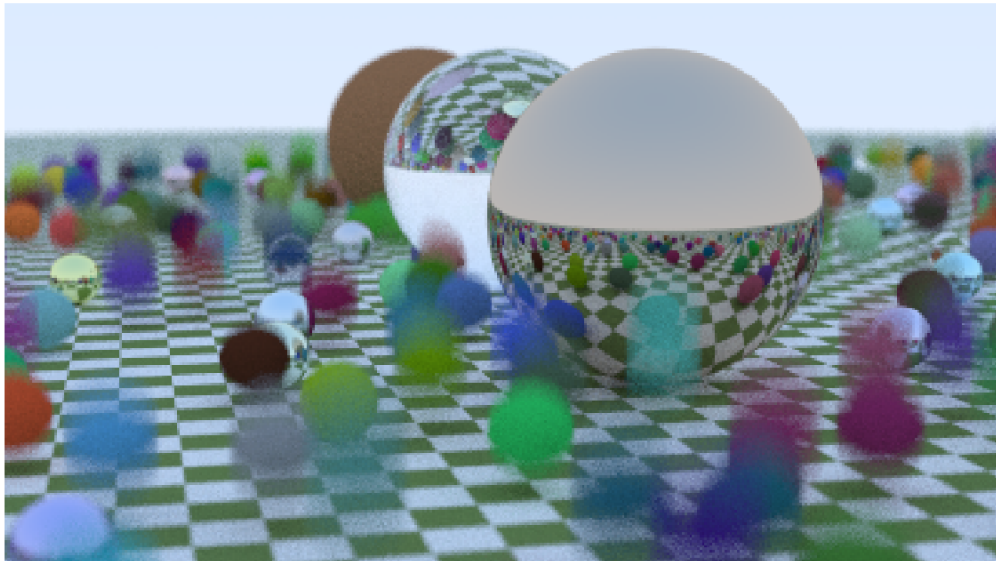


图 2: 格子地面上的球体

4.3. 渲染实心棋盘纹理

我们将在程序中添加第二个场景，并且随着本书内容的推进，之后还会添加更多场景。为了实现这一点，我们会设置一个 switch 语句，在特定运行时选择所需的场景。这是一种比较粗糙的方法，但我们尽量让事情保持简单，专注于光线追踪。在你自己的光线追踪器中，你可能希望使用不同的方法，比如支持命令行参数。

以下是我们为单个随机球体场景重构后的 main.cc 样子。将 main () 重命名为 bouncing_spheres ()，并添加一个新的 main () 函数来调用它：

```
#include "rtweekend.h"

#include "bvh.h"
#include "camera.h"
#include "hittable.h"
#include "hittable_list.h"
#include "material.h"
#include "sphere.h"
#include "texture.h"

int main() {
    void bouncing_spheres() {
        hittable_list world;

        auto ground_material = make_shared<lambertian>(color(0.5, 0.5, 0.5));
        world.add(make_shared<sphere>(point3(0, -1000, 0), 1000, ground_material));

        ...

        cam.render(world);
    }

    int main() {
        bouncing_spheres();
    }
}
```

清单 27: [main.cc] 主函数调度至所选场景

现在添加一个包含两个带格子图案的球体的场景。一个在另一个上面。

```
#include "rtweekend.h"

#include "bvh.h"
#include "camera.h"
#include "hittable.h"
#include "hittable_list.h"
#include "material.h"
#include "sphere.h"
#include "texture.h"

void bouncing_spheres() {
    ...
}

void checkered_spheres() {
    hittable_list world;

    auto checker = make_shared<checker_texture>(0.32, color(.2, .3, .1), color(.9, .9, .9));

    world.add(make_shared<sphere>(point3(0, -10, 0), 10, make_shared<lambertian>(checker)));
    world.add(make_shared<sphere>(point3(0, 10, 0), 10, make_shared<lambertian>(checker)));

    camera cam;

    cam.aspect_ratio      = 16.0 / 9.0;
    cam.image_width       = 400;
    cam.samples_per_pixel = 100;
    cam.max_depth         = 50;

    cam.vfov              = 20;
    cam.lookfrom = point3(13, 2, 3);
    cam.lookat  = point3(0, 0, 0);
    cam.vup     = vec3(0, 1, 0);

    cam.defocus_angle = 0;

    cam.render(world);
}

int main() {
    switch (2) {
        case 1: bouncing_spheres(); break;
        case 2: checkered_spheres(); break;
    }
}
```

清单 28: [main.cc] 两个带棋盘纹理的球体

我们得到了这样的结果：

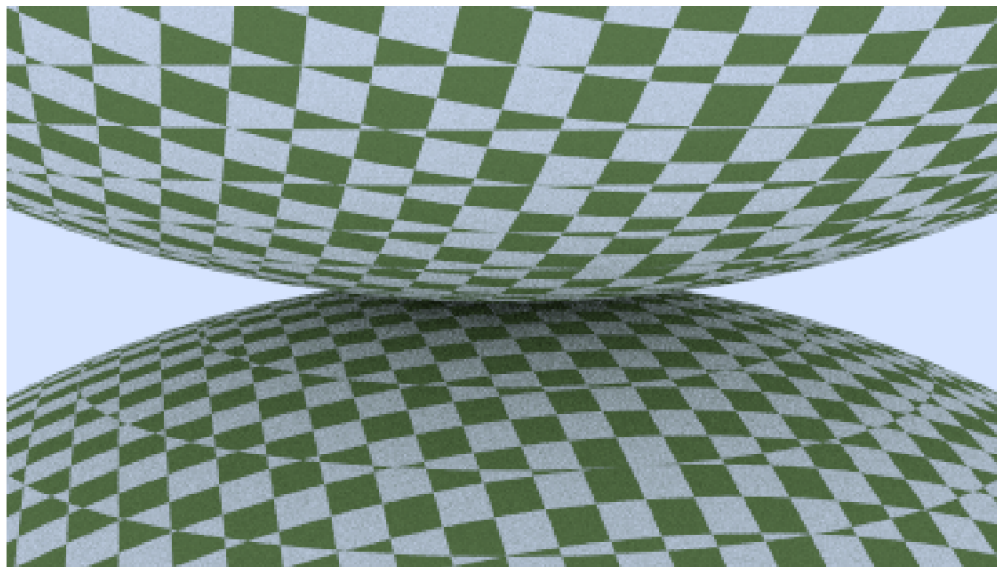


图 3：棋盘格球体

你可能会觉得结果看起来有点奇怪。由于 `checker_texture` 是一种空间纹理，我们实际上看到的是球体表面穿过三维棋盘格空间的样子。在很多情况下，这样的效果已经很理想，或者至少是足够的。但在其他许多情况下，我们确实希望在物体表面得到一致的效果。接下来就会介绍这种方法。

4.4. 球体的纹理坐标

恒色纹理不使用坐标。实体（或空间）纹理使用二维源图像上（或某些二维参数化空间中）位置的坐标。要实现这一点，我们需要一种任意的办法，但通常你希望覆盖整个表面，并且能够以某种合理的方式缩放、定向和拉伸二维图像。我们将从推导一种获取球体的 u 、 v 坐标的方案开始。空间中的点。现在是时候利用 u 、 v 纹理坐标了。这些坐标指定了如何找到三维物体表面上任意一点的 u 、 v 坐标。这种映射是完全的。

对于球体，纹理坐标通常基于某种形式的经度和纬度，即回到 $-X$ 球坐标。因此，我们在球坐标中计算 (θ, ϕ) ，其中 θ 是从下极点向上的角度（即从 $-Y$ 向上的角度），而 ϕ 是绕 Y 轴从 $-X$ 到 $+Z$ 、 $+X$ 再到 $-Z$ 的角度。

我们希望将 θ 和 ϕ 映射到纹理坐标 u 和 v ，每个坐标都在 $[0,1]$ 范围内，其中 $(u, v) = (0, 0)$ 映射到纹理的左下角。因此，从 (θ, ϕ) 到 (u, v) 的归一化将是：

$$u = \frac{\phi}{2\pi}$$

$$v = \frac{\theta}{\pi}$$

对应的笛卡尔坐标方程：要计算以原点为中心的单位球面上给定点的 θ 和 ϕ ，我们从以下内容开始

$$u = -\cos(\theta)$$

$$x = -\cos(\phi)\sin(\theta)$$

$$z = \sin(\phi)\sin(\theta)$$

`std::atan2()` 函数接受任何与正弦和余弦成比例的数对，并返回结果。由于 `<cmath>` 函数的强大功能，我们需要对这些方程讲行反转以求解 θ 和 ϕ 。

角度，我们可以传入 x 和 z 抵消) 来求解： $\sin(\theta)\phi$

$$\phi = \text{atan2}(z, -x)$$

`std::atan2()` 返回的值范围是 $-\pi$ 到 π ，但它们从 0 到 π ，然后翻转到 $-\pi$ 并且回到零。虽然这在数学上是正确的，但我们希望 u 的范围是从 π 到 0，而不是从 0 到 1，然后从 0 到 π 。幸运的是，

$$\text{atan2}(a, b) = \text{atan2}(-a, -b) + \pi$$

第二种表述产生的值从 0 连续到 2π 。因此，我们可以将 ϕ 计算为

$$\phi = \text{atan2}(-z, x) + \pi$$

θ 的推导更为直接：

$$\theta = \arccos(-u)$$

因此，对于球体， (u, v) 坐标计算是通过一个实用函数完成的，该函数接受以原点为中心的单位球面上的点以及 u 和 v 的分量

```
class sphere : public hittable {
...
private:
...

    static void get_sphere_uv(const point3& p, double& u, double& v) {
        // p: a given point on the sphere of radius one, centered at the origin.
        // u: returned value [0,1] of angle around the Y axis from X=-1.
        // v: returned value [0,1] of angle from Y=-1 to Y=+1.
        //      <1 0 0> yields <0.50 0.50>      <-1 0 0> yields <0.00 0.50>
        //      <0 1 0> yields <0.50 1.00>      < 0 -1 0> yields <0.50 0.00>
        //      <0 0 1> yields <0.25 0.50>      < 0 0 -1> yields <0.75 0.50>

        auto theta = std::acos(-p.y());
        auto phi = std::atan2(-p.z(), p.x()) + pi;

        u = phi / (2*pi);
        v = theta / pi;
    }
};
```

清单 29: [sphere.h] `get_sphere_uv` 函数

更新 `isphere::hit()` 函数，以使用此函数来更新命中记录的 UV 坐标。

```
class sphere : public hittable {
public:
    ...
    bool hit(const ray& r, interval ray_t, hit_record& rec) const override {
        ...

        rec.t = root;
        rec.p = r.at(rec.t);
        vec3 outward_normal = (rec.p - current_center) / radius;
        rec.set_face_normal(r, outward_normal);
        get_sphere_uv(outward_normal, rec.u, rec.v);
        rec.mat = mat;

        return true;
    }
    ...
};
```

清单 30: [sphere.h] 从命中点获取的球体 UV 坐标

我们的程序性实体纹理（如大理石纹理）。我们还可以读入图像，并使用二维纹理坐标来索引图像。根据撞击点 P ，我们计算出表面坐标 (u, v) 。然后，我们使用这些坐标来索引到 (u, v)

像素。这很麻烦，因为我们不希望在改变图像分辨率时还要修改代码。因此，图形学中最通用的非官方标准之一是使用纹理坐标而非图像像素坐标。这些只是某种形式的纹理位置：在图像中直接使用经过缩放的 (u, v) 的一种直接方法是将 u 和 v 取整为整数，并将其用作

图像中的分数位置。例如，对于 $N_x \times N_y$ 图像中的像素 (i, j) ，该图像

$$u = \frac{i}{N_x - 1}$$

$$v = \frac{j}{N_y - 1}$$

这只是一个部分位置。

4.5. 访问纹理图像数据

现在是时候创建一个用于存储图像的纹理类了。我将使用我最喜欢的图像工具 `stb_image`。它会将图像数据读取到一个 32 位浮点值数组中。这些数组只是打包的 RGB 值，每个分量的范围在 $[0, 1]$ 之间（从黑色到纯白色）。此外，图像是以线性色彩空间（伽马值 = 1）加载的，这是我们进行所有计算时所采用的色彩空间。

为了让加载我们的图像文件变得更加简单，我们提供了一个辅助类来管理所有这些操作：`rtw_image`。它提供了一个辅助函数——`pixel_data(int x, int y)`——用于获取每个像素的 8 位 RGB 字节值。下面的代码清单假设你已经将 `stb_image.h` 头文件复制到了一个名为 `external` 的文件夹中。请根据你的目录结构进行调整。


```

#ifndef RTW_STB_IMAGE_H
#define RTW_STB_IMAGE_H

// Disable strict warnings for this header from the Microsoft Visual C++ compiler.
#ifdef _MSC_VER
    #pragma warning (push, 0)
#endif

#define STB_IMAGE_IMPLEMENTATION
#define STBI_FAILURE_USERMSG
#include "external/stb_image.h"

#include <cstdlib>
#include <iostream>

class rtw_image {
public:
    rtw_image() {}

    rtw_image(const char* image_filename) {
        // Loads image data from the specified file. If the RTW_IMAGES environment
        variable is
        // defined, looks only in that directory for the image file. If the image was
        not found,
        // searches for the specified image file first from the current directory,
        then in the
        // images/ subdirectory, then the _parent's_ images/ subdirectory, and then
        _that_
        // parent, on so on, for six levels up. If the image was not loaded
        successfully,
        // width() and height() will return 0.

        auto filename = std::string(image_filename);
        auto imagedir = getenv("RTW_IMAGES");

        // Hunt for the image file in some likely locations.
        if (imagedir && load(std::string(imagedir) + "/" + image_filename)) return;
        if (load(filename)) return;
        if (load("images/" + filename)) return;
        if (load("../images/" + filename)) return;
        if (load("../../images/" + filename)) return;
        if (load("../../../images/" + filename)) return;
        if (load("../../../../images/" + filename)) return;
        if (load("../../../images/" + filename)) return;
        if (load("../../../../images/" + filename)) return;

        std::cerr << "ERROR: Could not load image file '" << image_filename <<
        "'.\n";
    }

    ~rtw_image() {
        delete[] bdata;
        STBI_FREE(fdata);
    }

    bool load(const std::string& filename) {

```

```

    // Loads the linear (gamma=1) image data from the given file name. Returns
    true if the
    // load succeeded. The resulting data buffer contains the three [0.0, 1.0]
    // floating-point values for the first pixel (red, then green, then blue).
    Pixels are
    // contiguous, going left to right for the width of the image, followed by
    the next row
    // below, for the full height of the image.

    auto n = bytes_per_pixel; // Dummy out parameter: original components per
    pixel
    fdata = stbi_loadf(filename.c_str(), &image_width, &image_height, &n,
    bytes_per_pixel);
    if (fdata == nullptr) return false;

    bytes_per_scanline = image_width * bytes_per_pixel;
    convert_to_bytes();
    return true;
}

int width() const { return (fdata == nullptr) ? 0 : image_width; }
int height() const { return (fdata == nullptr) ? 0 : image_height; }

const unsigned char* pixel_data(int x, int y) const {
    // Return the address of the three RGB bytes of the pixel at x,y. If there is
    no image
    // data, returns magenta.
    static unsigned char magenta[] = { 255, 0, 255 };
    if (bdata == nullptr) return magenta;

    x = clamp(x, 0, image_width);
    y = clamp(y, 0, image_height);

    return bdata + y*bytes_per_scanline + x*bytes_per_pixel;
}

private:
    const int      bytes_per_pixel = 3;
    float          *fdata = nullptr;    // Linear floating point pixel data
    unsigned char  *bdata = nullptr;    // Linear 8-bit pixel data
    int            image_width = 0;     // Loaded image width
    int            image_height = 0;    // Loaded image height
    int            bytes_per_scanline = 0;

    static int clamp(int x, int low, int high) {
        // Return the value clamped to the range [low, high).
        if (x < low) return low;
        if (x < high) return x;
        return high - 1;
    }

    static unsigned char float_to_byte(float value) {
        if (value <= 0.0)
            return 0;
        if (1.0 <= value)
            return 255;
        return static_cast<unsigned char>(256.0 * value);
    }

```

```
    }

    void convert_to_bytes() {
        // Convert the linear floating point pixel data to bytes, storing the
        // resulting byte
        // data in the `bdata` member.

        int total_bytes = image_width * image_height * bytes_per_pixel;
        bdata = new unsigned char[total_bytes];

        // Iterate through all pixel components, converting from [0.0, 1.0] float
        // values to
        // unsigned [0, 255] byte values.

        auto *bptr = bdata;
        auto *fptr = fdata;
        for (auto i=0; i < total_bytes; i++, fptr++, bptr++)
            *bptr = float_to_byte(*fptr);
    }
};

// Restore MSVC compiler warnings
#ifdef _MSC_VER
    #pragma warning (pop)
#endif

#endif
```

清单 31: [rtw_stb_image.h] rtw_image 辅助类

如果你使用 C 或 C++ 以外的语言编写实现，就需要找到（或编写）一个提供类似功能的图像加载库。

image_texture 类使用 rtw_image 类:

```
#include "rtw_stb_image.h"

...

class checker_texture : public texture {
    ...
};

class image_texture : public texture {
public:
    image_texture(const char* filename) : image(filename) {}

    color value(double u, double v, const point3& p) const override {
        // If we have no texture data, then return solid cyan as a debugging aid.
        if (image.height() <= 0) return color(0,1,1);

        // Clamp input texture coordinates to [0,1] x [1,0]
        u = interval(0,1).clamp(u);
        v = 1.0 - interval(0,1).clamp(v); // Flip V to image coordinates

        auto i = int(u * image.width());
        auto j = int(v * image.height());
        auto pixel = image.pixel_data(i,j);

        auto color_scale = 1.0 / 255.0;
        return color(color_scale*pixel[0], color_scale*pixel[1],
color_scale*pixel[2]);
    }

private:
    rtw_image image;
};
```

清单 32: [texture.h] 图像纹理类

4.6. 渲染图像纹理

我只是从网上随便找了一张地球地图 —— 任何标准投影图都能满足我们的需求。

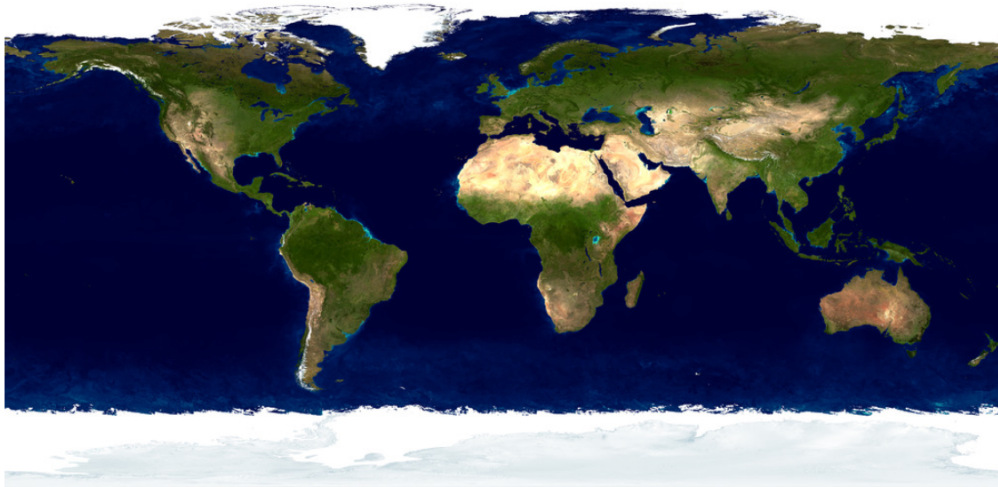


图 4: earthmap.jpg

以下是从文件读取图像并将其分配给漫反射材质的代码：

```
void earth() {
    auto earth_texture = make_shared<image_texture>("earthmap.jpg");
    auto earth_surface = make_shared<lambertian>(earth_texture);
    auto globe = make_shared<sphere>(point3(0,0,0), 2, earth_surface);

    camera cam;

    cam.aspect_ratio      = 16.0 / 9.0;
    cam.image_width       = 400;
    cam.samples_per_pixel = 100;
    cam.max_depth         = 50;

    cam.vfov              = 20;
    cam.lookfrom = point3(0,0,12);
    cam.lookat   = point3(0,0,0);
    cam.vup      = vec3(0,1,0);

    cam.defocus_angle = 0;

    cam.render(hittable_list(globe));
}

int main() {
    switch (3) {
        case 1: bouncing_spheres(); break;
        case 2: checkered_spheres(); break;
        case 3: earth(); break;
    }
}
```

清单 33: [main.cc] 使用地球纹理贴图进行渲染

我们开始看到所有颜色作为纹理所具备的部分力量 —— 我们可以为朗伯材质赋予任何类型的纹理，而朗伯材质本身无需知晓这一点。

如果照片中间出现一个大的青色球体，那么 `stb_image` 未能找到你的地球地图照片。程序会在与可执行文件相同的目录中查找该文件。请确保将地球图片复制到你的构建目录中，或者重新编写 `earth()` 函数以指向其他位置。



图 5：地球贴图球体

柏林噪声

为了获得看起来很酷的实体纹理，大多数人会使用某种形式的柏林噪声。这些噪声以其发明者肯·柏林的名字命名。柏林纹理不会返回像这样的白噪声：

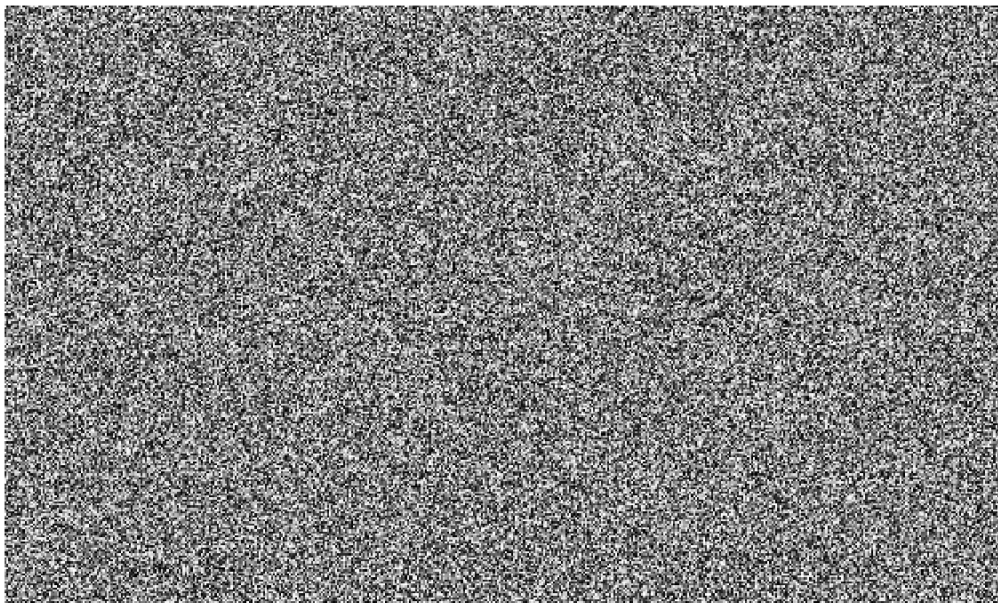


图 6：白噪声

相反，它返回的内容类似于模糊的白噪声：

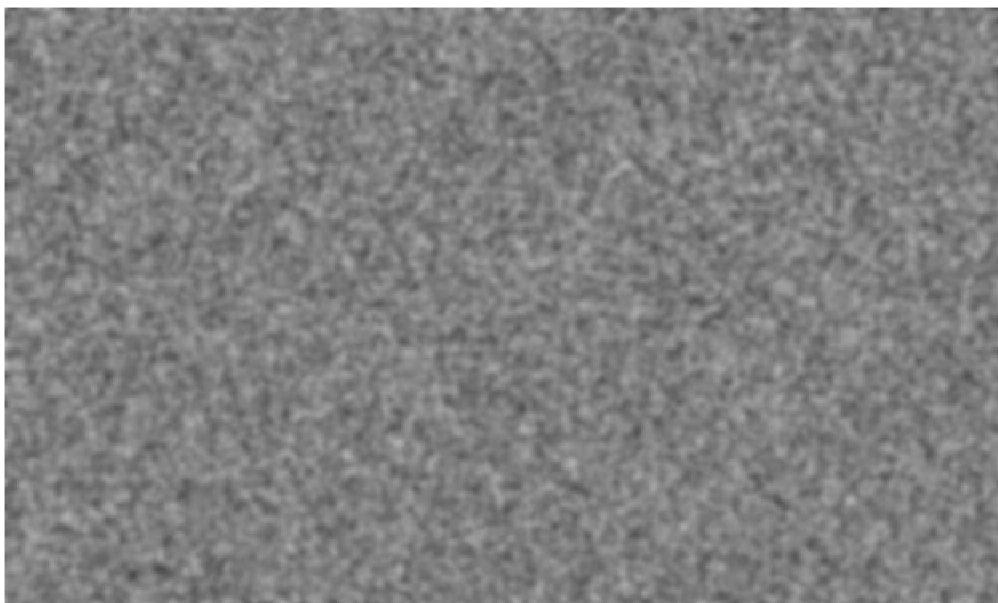


图 7：白噪声，模糊的

柏林噪声的一个关键特性是它具有可重复性：它接收一个三维点作为输入，并总是返回相同的类随机数。邻近的点会返回相似的数值。柏林噪声的另一个重要特性是它简单且快速，因此通常是通过一种取巧的方式实现的。我将基于安德鲁·肯斯勒的描述逐步构建这种取巧的实现方法。

5.1. 使用随机数块

我们可以用随机数的三维数组铺满整个空间，并分块使用它们。这样会得到一种块状效果，其中的重复性很明显：

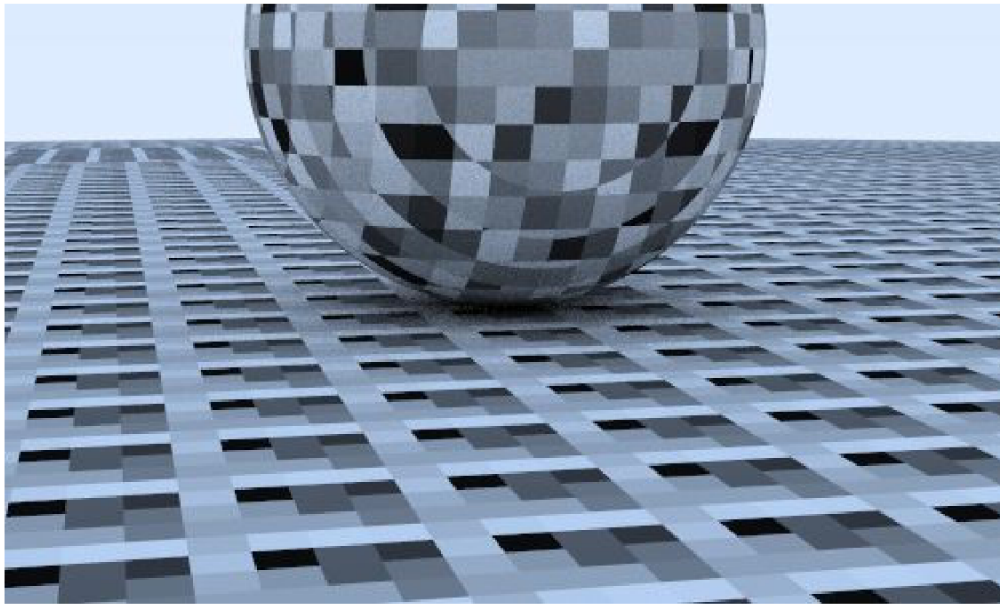


图 8：平铺随机图案

我们就用某种哈希方法来打乱这个，而不是平铺。这里有一些辅助代码来实现这一切：

```
#ifndef PERLIN_H
#define PERLIN_H

class perlin {
public:
    perlin() {
        for (int i = 0; i < point_count; i++) {
            randfloat[i] = random_double();
        }

        perlin_generate_perm(perm_x);
        perlin_generate_perm(perm_y);
        perlin_generate_perm(perm_z);
    }

    double noise(const point3& p) const {
        auto i = int(4*p.x()) & 255;
        auto j = int(4*p.y()) & 255;
        auto k = int(4*p.z()) & 255;

        return randfloat[perm_x[i] ^ perm_y[j] ^ perm_z[k]];
    }

private:
    static const int point_count = 256;
    double randfloat[point_count];
    int perm_x[point_count];
    int perm_y[point_count];
    int perm_z[point_count];

    static void perlin_generate_perm(int* p) {
        for (int i = 0; i < point_count; i++)
            p[i] = i;

        permute(p, point_count);
    }

    static void permute(int* p, int n) {
        for (int i = n-1; i > 0; i--) {
            int target = random_int(0, i);
            int tmp = p[i];
            p[i] = p[target];
            p[target] = tmp;
        }
    }
};

#endif
```

清单 34: [perlin.h] 一个柏林纹理类和函数

现在，如果我们创建一个实际的纹理，它接收 0 到 1 之间的这些浮点数并生成灰色：

```
#include "perlin.h"
#include "rtw_stb_image.h"

...
class noise_texture : public texture {
public:
    noise_texture() {}

    color value(double u, double v, const point3& p) const override {
        return color(1,1,1) * noise.noise(p);
    }

private:
    perlin noise;
};
```

清单 35: [texture.h] 噪声纹理